

Classify Your Agent with AiCMM — No Install, Just Prompt

You do not need to clone, build, or run AiCMM to use it. The Agent Capability Maturity Model is a framework of fixed rules and definitions, so any agentic CLI (GitHub Copilot CLI, Claude Code, Gemini CLI, Cursor, etc.) can apply it for you and produce a full AiCMM Agent Card — identical in shape and "look and feel" to the cards in the official catalog.

Get this one-pager: view it on the repo page — use the code block's copy button to grab the prompt — or download the Markdown / PDF.

On the AiCMM site the /classify-by-prompting page adds a one-click Copy Prompt button and a Download PDF link.

How to use it (under a minute):

1. Download this page (Markdown or PDF) and keep it handy — it is the prompt.
2. Open your favorite agentic CLI inside the repository of the agent you built.
3. Attach or paste the whole prompt block below, then replace the final AGENT TO CLASSIFY section with your agent's code, README, system prompt, and tool list — or simply write "Analyze the agent implemented in this repository."
4. The CLI returns a complete Agent Card: identity, avatar, the 12-dimension capability fingerprint, governance validation, and the derived Agency Qualification barometer — and saves three deliverables to disk: a styled HTML card, a print-ready PDF, and the machine-readable JSON.

The AiCMM Classifier Prompt (copy everything in the box)

```
SYSTEM ROLE
You are an AiCMM (Agent Capability Maturity Model) assessor. Produce a complete,
schema-accurate AiCMM Agent Card for the agent described under "AGENT TO CLASSIFY".
Score ONLY from observable evidence (source code, docs, prompts, tool lists, logs,
evals). Never inflate scores; if evidence is missing, score low and mark confidence
"low". Be consistent: the JSON scores, the governance results, and the Agency
Qualification MUST all be derived from the same 12 numbers.

=====
LEVEL 0 – 12 DIMENSIONS (fixed positions 0-11), each scored 0-5
-----
Cognitive Core:      0 autonomy  1 reasoning  2 memory  3 learning
```

;

```
f = clamp((avg12 - band[0]) / (band[1] - band[0]), 0, 0.96); needle = level + f
```

=====

OUTPUT – produce BOTH parts, in this order

PART 1 – Rendered Agent Card (human-readable, mirrors the AiCMM catalog card):

- Header: agent name, vendor, CATEGORY (digital | embodied | hybrid), one-line description.
- Avatar: archetype, tagline, 3-5 personality traits, top strengths, key limitations.
- Level 0 – Universal Capability Fingerprint: the one-line fingerprint string, then a table (dimension | pos | score 0-5 | confidence | one-line evidence). Show total /60 and average /5.
- Governance Validation: a table of all 7 rules (rule | constraint | PASS/FAIL/N/A | values) and overallCompliant.
- Agency Qualification: level, code, label, isAgent, Agency Index (0-100), needle, and a one-line rationale.

PART 2 – Agent Card JSON (schemaVersion "0.2.0"). Emit ONE JSON object with these keys:

```
agent { name, version, vendor, description, category, url }
avatar { archetype, tagline, personality[], visualTraits{ form, palette[], symbol, style }, strengths[], weaknesses[] }
capabilityProfile { _dimensionSchema:"level0-v0.2",
  <each of the 12 keys>: { position, score, confidence, evidence } }
operationalConstraints { domain, safetyClass, approvalRequirements[], regulatoryFrameworks[] }
standardsIntegration { a2a{ compatible, capabilities[] }, mcp{ role, connectedServers[], toolCount }, openai{ functionCallingCompatible } }
capabilityResume [ { version, date, schemaVersion:"0.2.0", scores{ 12 keys }, total, maxPossible:60, average, notes } ]
governanceValidation { rules:[ { rule, constraint, result, values } ], overallCompliant }
agencyQualification { position:12, dimension:"Agency Qualification", derived:true,
  level, code, label, isAgent, governancePass, index, needle, rationale }
tools[] skills[] plugins[] mcps[] (each a flat array of short strings)
agentRelationships { delegatesTo[], usedBy[], dependsOn[] }
assessmentMetadata { assessedBy, assessedDate, methodology:"AiCMM Level 0 v0.2 – evidence-based", evidenceSources[] }
```

Set category to digital/embodied/hybrid based on whether the agent acts in the physical world. Omit a field only if truly unknown.

PART 3 – Save the deliverables to disk (do this yourself with your file/tools;

do not just print paths). Create an ./aicmm-report/ folder and write:

1. aicmm-report/<agent-name>-agent-card.json – the PART 2 JSON, pretty-printed. Also copy it to examples/<agent-name>-agent-card.json if run inside the AiCMM repo.
2. aicmm-report/<agent-name>-agent-card.html – a self-contained, styled HTML page that renders PART 1: a hero header with the fingerprint string, an avatar block, an SVG radar chart of the 12 scores, the dimension table, the governance table, and an Agency Qualification gauge showing level/label/index/needle. Inline all CSS/SVG (no external assets) so it opens offline.
3. aicmm-report/<agent-name>-agent-card.pdf – a PDF of the same card. Generate it however your environment allows: render the HTML to PDF (headless browser / wkhtmltopdf / your PDF tool), or build it directly (e.g. Python fpdf2 / reportlab). If no PDF path is available, say so and leave the HTML + JSON.

After writing, OPEN the HTML page for the user and print the three saved file paths.

=====

AGENT TO CLASSIFY

<Paste your agent's code, README, system prompt, and tool list here – or write:

"Analyze the agent implemented in this repository.">

What you get back

Three files in `./aicmm-report/`, plus the opened HTML card:

File | What it is

`<agent>-agent-card.html` | A styled, self-contained card — hero + fingerprint, avatar, SVG radar chart, dimension table, governance table, and the Agency Qualification gauge. Opens offline.

`<agent>-agent-card.pdf` | A print-ready PDF of the same card, ready to share or attach to a review.

`<agent>-agent-card.json` | The machine-readable Agent Card (schema 0.2.0) you can drop straight into the catalog.

The card mirrors the exact shape and sections AiCMM renders natively: identity + avatar, the 12-dimension capability fingerprint, a governance pass/fail table, and the Agency Qualification verdict (with the 0–100 Agency Index) that tells you whether your system is a scripted automation, a reactive assistant, or a genuinely qualified agent (Basic → Humanoid).

Tips

- Point the CLI at real evidence (source files, prompts, eval logs) for higher-confidence scores.
- Ask for Level 1 domain scoring (e.g. Healthcare, Finance) if you deploy in a regulated domain.
- Validate your JSON against schemas/agent-card.schema.json.

Go further — use the repo, or help shape AiCMM

The prompt above is deliberately self-contained, but it is a portable snapshot of a living open-source framework. Clone the AiCMM repository when you want more than a one-shot classification:

- Deterministic scoring & validation — the Java engine computes governance and the Agency Qualification the same way every time (no LLM variance), via `mvn build`, the CLI (``aicmm validate | score | classify``), and the REST API.
- Interactive site & catalog — a web UI with live radar rendering, avatars, and a searchable catalog of published cards (`java -jar aicmm-site/...`).
- MCP server — expose AiCMM as tools to Copilot / Claude / Gemini (`--mcp`), so your CLI calls the real scorer instead of approximating it.
- The full ruleset as source of truth — the 12 dimensions, 7 governance rules, and the Agency ladder are locked by tests in `aicmm-core`; read them to justify every score.

Help shape the project. AiCMM is MIT-licensed and community-driven. Contributions are welcome — publish your Agent Card to the catalog (examples/`<agent>-agent-card.json`),

propose a new Level 1 domain, refine dimension rubrics or governance rules, or improve the docs. See CONTRIBUTING.md, open an issue, or send a PR. Your real-world classifications make the model sharper for everyone.

AiCMM is open source (MIT). This prompt-only workflow requires no installation — the repository is there when you want deterministic scoring, tooling, or a seat at the table.